MODULE 18 · DOMAIN 4 - MONITORING AND LOGGING

Analysing and Alarming

Logs Insights and its three query languages, anomaly detection, alarms that do not cry wolf, composite alarms, Contributor Insights, dashboards and Athena.

DOMAIN 4 of 6 EXAM WEIGHT 15% MODULE 18 of 34

admin@sthithapragna.com

An alarm is a **promise to wake someone**. If it fires for something that does not matter, people learn to ignore it, and then it fails when it does matter.

admin@sthithapragna.com

Which tool answers which question?

THE QUESTION	TOOL
What happened in these logs?	CloudWatch Logs Insights
What is happening right now?	Live Tail
Who or what is causing most of it?	Contributor Insights
Is this normal for a Tuesday at 9?	Anomaly detection
What happened over the last year, in S3?	Athena
What do customers actually experience?	Synthetics and RUM

ON THE EXAM

The data's location decides a lot: CloudWatch Logs means Logs Insights; S3 means Athena.

CloudWatch Logs Insights

- Query log groups interactively: filter, parse, aggregate, sort, and chart results over time.
- Since December 2024, three languages: Logs Insights QL, **OpenSearch SQL** with joins, and **OpenSearch PPL**.
- **Field indexes** reduce the data a query scans, so filtered queries run faster and cost less.
- Up to 50 log groups chosen one by one; queries time out after 60 minutes.

> ON THE EXAM
>
> Queries can be saved, added to dashboards, and generated from a plain-English description.

A Logs Insights query, read line by line

```
1  fields @timestamp, path, status, duration
2  | filter status >= 500
3  | stats count(*) as errors, avg(duration) as avgMs by bin(5m), path
4  | sort errors desc
5  | limit 20
```

2 Keep only server errors.

3 Count them, with average duration, per path, in five-minute buckets.

ON THE EXAM

stats … by bin() turns a query into a time series you can chart or add to a dashboard.

Letting CloudWatch notice for you

- **Live Tail** streams matching log events in near real time during an incident.
- **Pattern analysis** groups similar log lines, so new kinds of error stand out.
- **Log anomaly detection** learns a log group's normal patterns and flags unusual ones.
- **Metric anomaly detection** learns hourly and weekly seasonality, producing an expected band.

ON THE EXAM

No one knows what to search for: pattern analysis and log anomaly detection surface the unfamiliar.

The anatomy of an alarm

SETTING	WHAT IT CONTROLS
Metric and statistic	What is measured: Average, Sum, Maximum, or a percentile such as p99
Period	How long each data point covers
Threshold or band	A fixed value, or an anomaly detection band
Evaluation periods	How many recent data points are considered
Datapoints to alarm	How many of those must breach: M out of N
Missing data treatment	What a gap in the data means

ON THE EXAM

Latency alarms on Average hide the slowest users. Use p90 or p99 for user experience.

Stopping alarms that twitch

M OUT OF N DATAPOINTS

Sustained, but still fast

- 3 of 5 one-minute points must breach
- Ignores single spikes
- Still reacts within minutes
- Tolerates one odd point in the run

A LONGER PERIOD

Smoother, but slower

- One 15-minute data point
- Averages spikes away
- Reacts only after 15 minutes
- Can hide short, severe outages

THE TIEBREAKER

Pages for brief spikes that clear themselves: M out of N. A longer period trades away detection speed.

Treating missing data

SETTING	A GAP IS TREATED AS	USE WHEN
`missing (default)`	Not evaluated; may become INSUFFICIENT_DATA	Data should always be present
`notBreaching`	Good	Metric is only published when something goes wrong
`breaching`	Bad	Silence itself means failure, such as a heartbeat
`ignore`	Keeps the current state	Sparse metrics where state should persist

ON THE EXAM

An error-count metric published only when errors occur: treat missing data as notBreaching.

Kinds of alarm

- **Static threshold**: simple, right when 'bad' is a fixed number, such as disk 90% full.
- **Anomaly detection**: alarms outside the learned band, right for seasonal traffic.
- **Metric math**: alarm on an expression, such as errors divided by requests.
- **Metrics Insights** query alarms: alarm on a SQL-style query across many resources.

> ON THE EXAM
>
> Error count rises with traffic; error rate does not. Alarm on the ratio with metric math.

Composite alarms

- Combine alarms with **AND**, **OR** and **NOT** into one alarm with one action.
- Put actions on the composite; leave the child alarms without notifications.
- An **actions suppressor** silences a composite, for example during deployments or when an upstream alarm explains it.
- Result: one page for the cause, not forty for its symptoms.

> ON THE EXAM
>
> Alert storms during a database outage: composite alarms, with suppression from the database alarm.

What an alarm can do

ACTION	EXAMPLE
Notify through SNS	Email, chat, or a paging tool
Invoke a Lambda function	Custom response, directly from the alarm
EC2 action	Recover, reboot, stop or terminate the instance
Auto Scaling action	Trigger a scaling policy
Systems Manager	Open an OpsItem or an incident
EventBridge	Every state change is an event: route it anywhere

ON THE EXAM

EventBridge sees every alarm state change, so it can start runbooks, pipelines or rollbacks.

Contributor Insights

- Rules over structured logs find the **top contributors**: IP addresses, users, URLs, keys.
- Results are time series, so you can graph and alarm on them.
- Built-in rules exist for VPC Flow Logs and DynamoDB, including hot partition keys.
- A metric per client would explode cardinality. Contributor Insights avoids that.

> ON THE EXAM
>
> Which clients cause most throttling or errors, continuously: Contributor Insights, not a metric per client.

Dashboards

- Widgets for metrics, alarms, Logs Insights queries, and text or links to runbooks.
- One dashboard can show metrics from **several accounts and Regions**.
- Automatic dashboards appear per service; share dashboards with people outside AWS.
- Dashboards need someone to look. Anything urgent belongs in an alarm.

> ON THE EXAM
>
> A single view of a service running in three Regions and two accounts: a cross-account, cross-Region dashboard.

Querying logs in S3 with Athena

- Athena runs SQL directly over files in S3: ALB, CloudTrail, VPC Flow and S3 access logs.
- You pay per terabyte scanned, so scanning less is both faster and cheaper.
- **Partition** by date, ideally with partition projection, so queries read only relevant days.
- Convert to a columnar format such as **Parquet**, so queries read only the needed columns.

> ON THE EXAM
>
> Athena queries on years of logs are slow and expensive: partition by date and convert to Parquet.

Logs Insights or Athena?

ATHENA

Logs in S3

- Long-term, low-cost storage
- SQL, joins across data sets
- Tuned with partitions and Parquet
- Best for months or years of history

LOGS INSIGHTS

Logs in CloudWatch

- No set-up: query any log group
- Great for recent, operational data
- Charts and dashboard widgets
- Retention costs more than S3

THE TIEBREAKER

Investigating last hour's incident: Logs Insights. A year-long analysis of archived logs: Athena.

Seeing what customers see

- **Synthetics canaries** run scripted journeys on a schedule, from outside, as in module 4.
- **CloudWatch RUM** collects real users' page load times, errors and sessions from their browsers.
- Canaries catch breakage even with no traffic. RUM shows what real users actually suffered.
- Both feed metrics you can alarm on, alongside your server-side metrics.

> ON THE EXAM
>
> 'Users in one country report slow pages our metrics do not show': RUM captures real client-side experience.

> A CPU alarm on a web fleet uses one-minute periods, one evaluation period and a threshold of 80%. It pages the on-call engineer several times a night.
>
> Each time, CPU spikes for a single minute and recovers. The team wants pages only when high CPU is sustained, but still within about five minutes.

What should the DevOps engineer change?

A. Require 3 of 5 datapoints to breach, keeping one-minute periods

B. Raise the threshold to 95%

C. Change the period to one hour

D. Replace the alarm with an anomaly detection alarm

◆ PAUSE AND DECIDE

What should the DevOps engineer change?

A 3 of 5 datapoints

A single-minute spike never reaches three; sustained load alarms within five minutes.

B Higher threshold

Spikes can still exceed 95%, and real sustained problems at 90% are now missed.

C One-hour period

Calm, but detection is now an hour late. Fails 'within about five minutes'.

D Anomaly detection

A sudden spike is anomalous too. It changes the threshold, not the twitchiness.

THE SIGNAL

Spikes that self-correct, detection still needed in minutes: M out of N datapoints.

> When the shared database fails, about 40 alarms fire across the web, application and worker tiers, and each one pages the on-call engineer.
>
> The team wants a single page that identifies the database as the cause, while keeping the individual alarms for diagnosis.

What should the DevOps engineer do?

A Add SNS subscription filter policies to drop duplicate messages

B Disable the application and worker alarms

C Raise every threshold so fewer alarms fire

D Create composite alarms that carry the notifications, remove actions from child alarms, and use the database alarm as an actions suppressor

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A SNS filtering

The messages are not duplicates: 40 different alarms. Filtering cannot tell cause from symptom.

B Disable alarms

Loses diagnosis and blinds you to tier failures unrelated to the database.

C Raise thresholds

Fewer alarms, but also later detection of genuine problems everywhere.

D Composite alarms with suppression

One page for the cause; the symptom alarms stay visible but silent.

THE SIGNAL

Alert storm from one root cause: composite alarms and an actions suppressor.

> An API's request count varies strongly by hour and weekday. A static alarm set low fires every night; set high, it missed a real drop in traffic during business hours.
>
> The team wants an alarm that fires when traffic is unusually low or high for that time of the week.

What should the DevOps engineer use?

A. A static alarm with a threshold halfway between night and day traffic

B. An anomaly detection alarm on request count, alarming outside the band

C. A static alarm on the p99 of request count

D. A scheduled Lambda function that changes the alarm threshold every hour

◆ PAUSE AND DECIDE

What should the DevOps engineer use?

A **Halfway threshold**

Wrong at both ends: false alarms at night, missed drops by day.

B **Anomaly detection band**

Learns hourly and weekly seasonality, and alarms when traffic leaves the expected band either way.

C **p99 of a count**

Percentiles describe distributions of values like latency, not a request count. Still static.

D **Hourly threshold Lambda**

Hand-builds a crude version of what anomaly detection learns automatically.

THE SIGNAL

'Unusual for this time of the week': anomaly detection. No single number can be right.

> An application publishes a PaymentFailures metric only when a payment fails. An alarm fires when the sum exceeds 5 in 5 minutes.
>
> Most of the time there are no failures, so no data points arrive, and the alarm sits in INSUFFICIENT_DATA. The team wants it to show OK when nothing is failing.

What should the DevOps engineer change?

A Set the alarm to treat missing data as notBreaching

B Set the alarm to treat missing data as breaching

C Add a cron job that publishes a zero every minute

D Shorten the period to one minute

◆ PAUSE AND DECIDE

What should the DevOps engineer change?

A notBreaching

Silence means no failures, so gaps count as good and the alarm shows OK.

B breaching

The opposite: the alarm would fire whenever payments are healthy.

C Publish zeros

Works, but adds a job to run and pay for, to express what one setting says.

D Shorter period

More frequent gaps, same problem.

THE SIGNAL

Metric published only on failure: silence is good, so notBreaching. Heartbeat metric: silence is bad, so breaching.

> An API writes structured JSON logs that include the caller's account ID and the response status. Throttling errors sometimes rise sharply.
>
> Operators need to see, continuously and as a graph, which callers cause the most throttling, without writing a query each time and without creating a metric per caller.

What should the DevOps engineer use?

A. A saved Logs Insights query that operators run during incidents

B. A metric filter with the caller's account ID as a dimension

C. AWS X-Ray service maps

D. A Contributor Insights rule on the log group, keyed on caller and filtered to throttling

◆ PAUSE AND DECIDE

What should the DevOps engineer use?

A **Saved query**

Must be run each time. Not continuous.

B **Metric filter per caller**

One metric per caller. the cardinality the requirement ruled out.

C **X-Ray service maps**

Show services and latency, not a ranking of callers.

D **Contributor Insights**

Continuously ranks top callers as a time series, with no per-caller metrics.

THE SIGNAL

Continuous top-N by a high-cardinality key: Contributor Insights.

> Three years of ALB access logs, several terabytes, sit in S3 as compressed text files under one prefix. Security analysts query them with Athena.
>
> Queries about a single week take many minutes and scan the whole data set, and the Athena bill is rising.

What should the DevOps engineer do?

A Load the logs into an Amazon RDS database for querying

B Move the logs into CloudWatch Logs and use Logs Insights

C Partition the data by date with partition projection, and convert it to Parquet

D Increase the Athena workgroup's data scan limit

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A Load into RDS

A database to size, load and run, for terabytes of append-only logs.

B Move to CloudWatch Logs

Re-ingesting terabytes costs far more, and CloudWatch storage exceeds S3's.

C Partitions and Parquet

A week's query reads a week's partitions, and only the columns it needs.

D Raise the scan limit

Lets queries scan even more. The opposite of the fix.

THE SIGNAL

Athena slow and expensive: scan less. Partition by what you filter on, and store columns.

Module 18 in five lines

- Logs Insights for CloudWatch Logs, now in SQL and PPL too; Athena for S3, tuned by partitions and Parquet.
- Alarms: M out of N for spikes, anomaly bands for seasonality, metric math for rates.
- Missing data: notBreaching when silence is good, breaching when silence is bad.
- Composite alarms and suppression turn alert storms into one page.
- Contributor Insights for top-N; Synthetics and RUM for the customer's view.

ON THE EXAM

Next, module 19: tracing and distributed observability. X-Ray, OpenTelemetry, Application Signals and Grafana.